

The Amazon Forest

THE AMAZON FOREST *A 2-Tier Cloud Architecture on AWS*

Training Program by: Cybersteps

Cohort: 12/25

Name: E-Itohan

Date: 30.09.2026

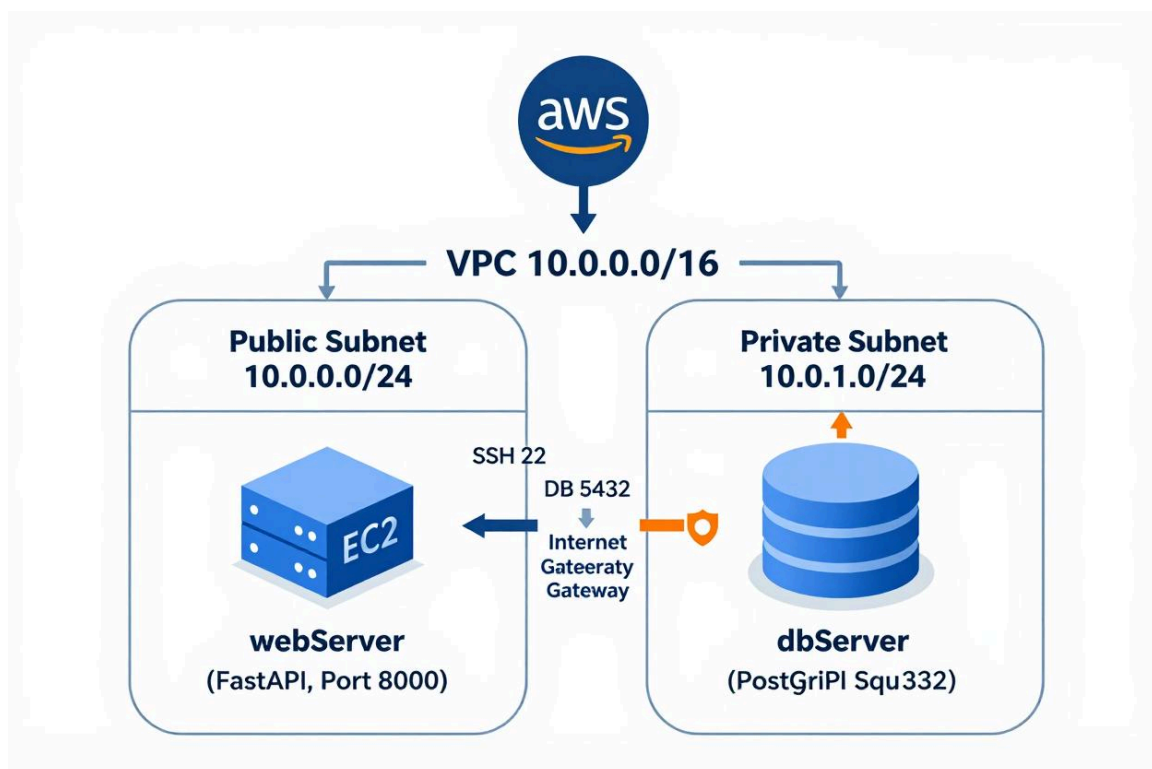


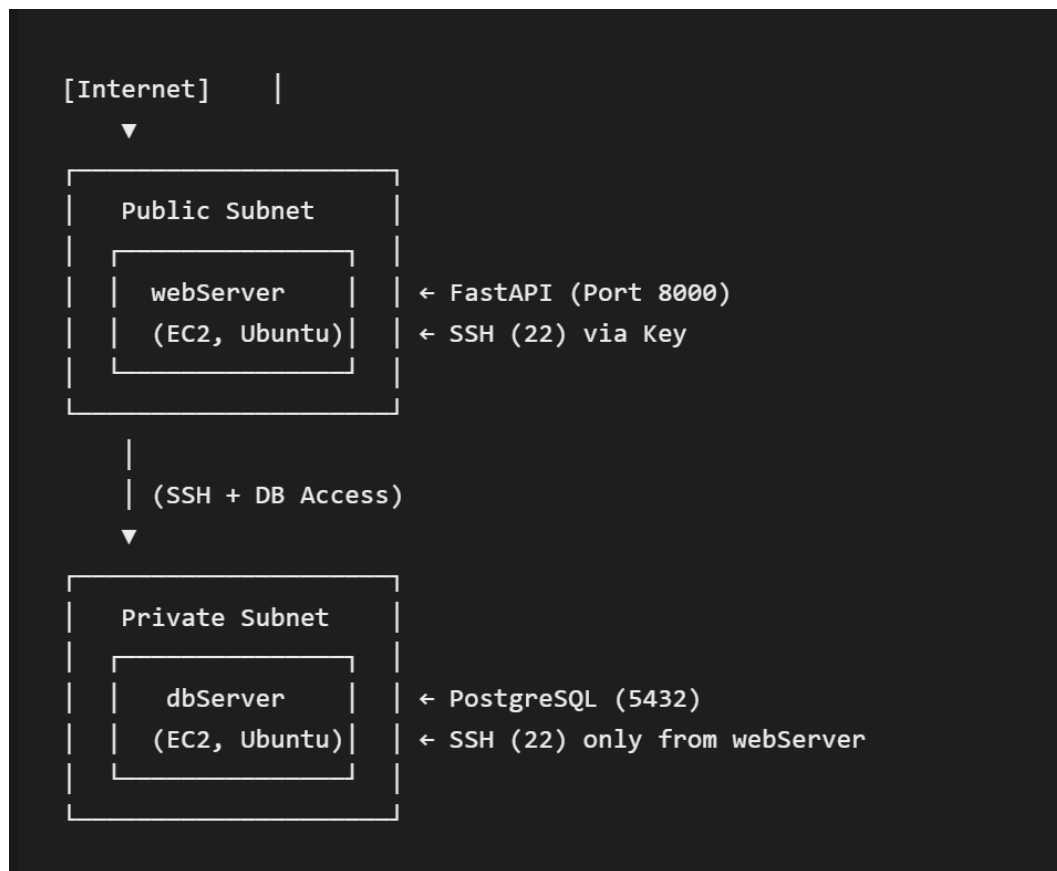
TABLE OF CONTENTS

1. Overview
 2. Architecture Design
 3. Implementation on AWS
 4. Server Setup & Configuration
 5. Web Application Deployment
 6. AWS vs Azure: Key Differences
 7. Challenges & Troubleshooting
 8. Lessons Learned
-

1. OVERVIEW

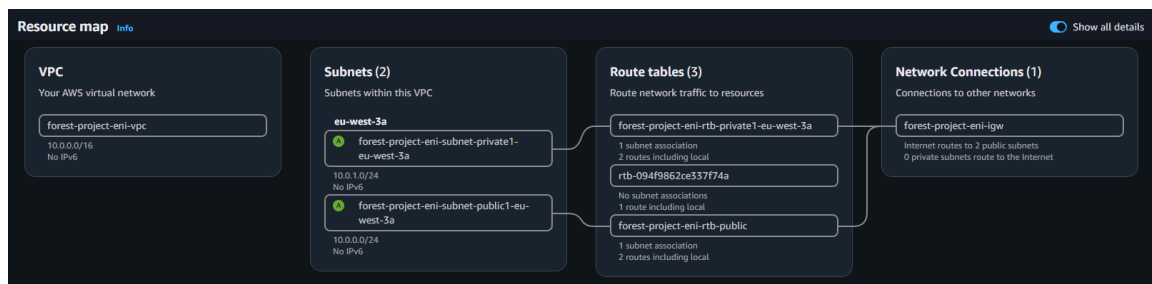
This project implements a **2-tier architecture on AWS**, consisting of an internet-facing FastAPI application server (Tier 1) and a private PostgreSQL database server (Tier 2). The goal was to translate a project previously built on Azure into AWS, in order to understand the conceptual differences between cloud providers while applying the same architectural principles.

2. ARCHITECTURE DESIGN



Architecture schema

VPC with two subnets:



VPC Map

Subnet	Placement	Purpose	Access
Public subnet	Tier 1	FastAPI application server (webServer)	Internet-facing
Private subnet	Tier 2	PostgreSQL database (dbServer)	Internal only

Security rules — public subnet:

- SSH (port 22) — key-based access only

- HTTP (port 8000) — reachable from the internet

Security group rule ID	IP version	Type	Protocol	Port range	Source
sgr-0ce1e5db4c24a0e3f	IPv4	Custom TCP	TCP	8000	0.0.0.0/0
sgr-046759d08b68ba85f	IPv4	SSH	TCP	22	0.0.0.0/0

Security rules for public subnet

Security rules — private subnet:

- SSH (port 22) — reachable **only** from the API machine, using the same key
- PostgreSQL (port 5432) — reachable only from `webServer`

Security group rule ID	IP version	Type	Protocol	Port range	Source
sgr-05dea7df936c1cf07	-	PostgreSQL	TCP	5432	webServer
sgr-0206b2993925743fd	-	SSH	TCP	22	API machine

Security rules for private subnet

Instances:

Instance	Role	Subnet
<code>webServer</code>	FastAPI application (Tier 1)	Public
<code>dbServer</code>	PostgreSQL database (Tier 2)	Private

3. IMPLEMENTATION ON AWS

3.1 AWS Portal Setup

1. Create a **VPC** with two subnets (public and private)
2. Configure security rules per subnet (see Section 2)
3. Launch both EC2 instances (Ubuntu)
4. Allocate an **Elastic IP** for internet access on `dbServer`

3.2 Connecting to the Servers

Connect from host to `webServer` :

```
ssh -i [path/to/key] ubuntu@[web-server-ip]
```

```
*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-10-0-0-176:~$ whoami
ubuntu
ubuntu@ip-10-0-0-176:~$ |
```

Connection to `webServer`

Copy the key from host to `webServer` so the same key can be reused for the database connection:

```
scp -i [path/to/key] [key-file] ubuntu@[web-server-ip]:[destination]
```

```
ubuntu@ip-10-0-0-176:~$ ls
key.pem
ubuntu@ip-10-0-0-176:~$
```

Key Copy

Fix permissions on the copied key:

```
chmod 600 key.pem
```

```
ubuntu@ip-10-0-1-20:~$ whoami
ubuntu
ubuntu@ip-10-0-1-20:~$
```

Connection verification

Connect from `webServer` to `dbServer` :

```
ssh -i key.pem ubuntu@[db-server-ip]
```

4. SERVER SETUP & CONFIGURATION

4.1 Installing PostgreSQL on `dbServer`

Note: An Elastic IP allocation was necessary to get internet access on the database instance during installation.

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib
sudo systemctl enable --now postgresql
```

4.2 Database, Table, User and Password Creation

Created the database, table, user, and password on `dbServer`.

```
ubuntu@ip-10-0-1-216:~$ sudo -u postgres psql -c "CREATE DATABASE learning_journal;"
CREATE DATABASE
ubuntu@ip-10-0-1-216:~$ sudo -u postgres psql -c "CREATE USER user123 WITH ENCRYPTED PASSWORD '
ord123';"
CREATE ROLE
ubuntu@ip-10-0-1-216:~$ sudo -u postgres psql -c "GRANT ALL PRIVILEGES ON DATABASE learning_jou
TO user123;"
GRANT
ubuntu@ip-10-0-1-216:~$ sudo -u postgres psql -d learning_journal
psql (18.6 (Ubuntu 18.6-0ubuntu0.26.04.1))
Type "help" for help.

learning_journal=# CREATE TABLE IF NOT EXISTS entries (
    id VARCHAR PRIMARY KEY,
    data JSONB NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL
);
CREATE TABLE
learning_journal=# CREATE INDEX IF NOT EXISTS idx_entries_created_at ON entries(created_at);
CREATE INDEX IF NOT EXISTS idx_entries_data_gin ON entries USING GIN (data);
GRANT USAGE, CREATE ON SCHEMA public TO user123;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO user123;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO user123;
CREATE INDEX
CREATE INDEX
GRANT
```

Database Configuration

4.3 PostgreSQL Configuration

`/etc/postgresql/*/main/postgresql.conf` — line 60 changed to listen on all addresses:

```
listen_addresses = '*'
```

```

#-----
# CONNECTIONS AND AUTHENTICATION
#-----

# - Connection Settings -
listen_addresses = '*'           # what IP address(es) to listen on;
                                  # comma-separated list of addresses;
                                  # defaults to 'localhost'; use '*' for all
                                  # (change requires restart)
port = 5432                      # (change requires restart)
max_connections = 100            # (change requires restart)
#reserved_connections = 0        # (change requires restart)
#superuser_reserved_connections = 3 # (change requires restart)
unix_socket_directories = '/var/run/postgresql' # comma-separated list of directories
                                  # (change requires restart)
#unix_socket_group = ''          # (change requires restart)
#unix_socket_permissions = 0777 # begin with 0 to use octal notation
                                  # (change requires restart)
#bonjour = off                   # advertise server via Bonjour
                                  # (change requires restart)
#bonjour_name = ''              # defaults to the computer name
                                  # (change requires restart)

# - TCP settings -

```

postgresql.conf

/etc/postgresql/*/main/pg_hba.conf — added:

host all all 10.0.0.0/24 scram-sha-256

```

GNU nano 8.7.1 /etc/postgresql/18/main/pg_hba.conf
09
10
11 # DO NOT DISABLE!
12 # If you change this first entry you will need to make sure that the
13 # database superuser can access the database using some other method.
14 # Noninteractive access to all databases is required during automatic
15 # maintenance (custom daily cronjobs, replication, and similar tasks).
16 #
17 # Database administrative login by Unix domain socket
18 local all postgres peer
19
20 # TYPE DATABASE USER ADDRESS METHOD
21
22 # "local" is for Unix domain socket connections only
23 local all all peer
24 # IPv4 local connections:
25 host all all 127.0.0.1/32 scram-sha-256
26 # IPv6 local connections:
27 host all all ::1/128 scram-sha-256
28 # Allow replication connections from localhost, by a user with the
29 # replication privilege.
30 local replication all peer
31 host replication all 127.0.0.1/32 scram-sha-256
32 host replication all ::1/128 scram-sha-256
33 host all all 10.0.0.0/24 scram-sha-256
34

```

pg_hba.conf

4.4 Connection Verification

Verified the connection from **webServer**:

nc -vz [db-server-ip] [port]

```
ubuntu@ip-10-0-0-190:~$ nc -vz 10.0.1.216 5432
Connection to 10.0.1.216 5432 port [tcp/postgresql] succeeded!
ubuntu@ip-10-0-0-190:~$
```

dbServer connection verification

5. WEB APPLICATION DEPLOYMENT

5.1 Repository and Environment

```
git clone -b reference https://github.com/roman-cybersteps/learningsteps.git
cd learningsteps/
mv .env-sample .env
```

Configured the virtual environment, adding the database name, database URL, user, and password created on `dbServer`.

5.2 Dependencies

```
sudo apt install -y python3 python3-pip python3-venv git
```

5.3 Running the Application

```
source venv/bin/activate
sudo ./start.sh
```

The web app is available at: `http://[webapp-public-ip]:8000/docs`

LearningSteps API 0.1.0 OAS 3.1

/openapi.json

A simple learning journal API for tracking daily work, struggles, and intentions

default			^
GET	/entries	Get All Entries	▼
POST	/entries	Create Entry	▼
DELETE	/entries	Delete All Entries	▼
GET	/entries/{entry_id}	Get Entry	▼
PATCH	/entries/{entry_id}	Update Entry	▼
DELETE	/entries/{entry_id}	Delete Entry	▼

Desktop 5

Web app

5.4 Verification

Verified the deployment with `curl` from the terminal, plus GET and POST requests on `webServer`.

Curl

```
curl -X 'GET' \
'http://13.38.92.119:8000/entries' \
-H 'accept: */*'
```

Request URL

```
http://13.38.92.119:8000/entries
```

Server response

Code	Details
200	Response body <pre>{ "entries": [{ "id": "68acbd9e-0e49-4831-92cd-d0e8e82770a2", "work": "Studied FastAPI and built my first API endpoints", "struggle": "Understanding async/await syntax and when to use it", "intention": "Practice PostgreSQL queries and database design", "created_at": "2026-09-29T16:41:31.802819+00:00", "updated_at": "2026-09-29T16:41:31.802819+00:00" }], "count": 1 }</pre> Response headers

GET-request & server response on web app

```
ubuntu@ip-10-0-0-190:~$ curl -X 'GET' \
'http://13.38.92.119:8000/entries' \
-H 'accept: */*'
{"entries": [], "count": 0}ubuntu@ip-10-0-0-190:~$ curl -X 'GET' 'http://13.38.92.119:8000/entries'
-H 'accept: */*'
{"entries": [{"id": "68acbd9e-0e49-4831-92cd-d0e8e82770a2", "work": "Studied FastAPI and built my first API endpoints", "struggle": "Understanding async/await syntax and when to use it", "intention": "Practice PostgreSQL queries and database design", "created_at": "2026-09-29T16:41:31.802819+00:00", "updated_at": "2026-09-29T16:41:31.802819+00:00"}], "count": 1}ubuntu@ip-10-0-0-190:~$
```

Curl on terminal

```
WARNING: Invalid HTTP Request received.
INFO: 159.26.104.105:48340 - "GET /docs HTTP/1.1" 200 OK
INFO: 159.26.104.105:48340 - "GET /openapi.json HTTP/1.1" 200 OK
2026-09-29 16:37:49,484 - journal - INFO - Fetching all entries
INFO: 159.26.104.105:51310 - "GET /entries HTTP/1.1" 200 OK
2026-09-29 16:40:55,127 - journal - INFO - Fetching all entries
INFO: 13.38.92.119:48754 - "GET /entries HTTP/1.1" 200 OK
2026-09-29 16:41:31,802 - journal - INFO - Creating entry
INFO: 159.26.104.105:19406 - "POST /entries HTTP/1.1" 200 OK
2026-09-29 16:41:40,187 - journal - INFO - Fetching all entries
INFO: 159.26.104.105:9264 - "GET /entries HTTP/1.1" 200 OK
2026-09-29 16:42:51,807 - journal - INFO - Fetching all entries
INFO: 13.38.92.119:38054 - "GET /entries HTTP/1.1" 200 OK
```

GET & POST request on webServer

6. AWS VS AZURE: KEY DIFFERENCES

Feature	AWS	Azure
Virtual network	VPC (Virtual Private Cloud)	VNet (Virtual Network)
Subnet scope	Bound to a single Availability Zone	Regional construct, can span multiple AZs
Security model	Security Groups (stateful) + Network ACLs (stateless)	NSGs (stateful) + Application Security Groups
Public IP persistence	Elastic IP	Public IP (static)
Private subnet internet access	NAT Gateway	NAT Gateway
VM operating model	EC2 instances	Virtual Machines (VMs)
Naming conventions	Service-specific names (EC2, VPC, SG)	Resource-group oriented, more descriptive names

Takeaway: The architectural concepts are identical across providers, what changes is the naming convention, the console workflow, and details like subnet-to-AZ binding in AWS versus regional subnets in Azure. I will also note, that the speed of deployment on AWS felt faster as on Azure, sometimes less steps were needed on AWS to achieve the same goal.

7. CHALLENGES & TROUBLESHOOTING

7.1 Internet Access on the Database Instance

Getting internet on the database instance was initially impossible. **Solution:** attach an Elastic IP to obtain a public IP temporarily, install PostgreSQL, then release the Elastic IP afterward so costs do not increase.

7.2 Python Virtual Environment Issue

After cloning the repository, creating the environment on `webServer` failed. The cause: the `venv` symlinked to the generic `/usr/bin/python3` instead of `/usr/bin/python3.14`. **Solution:** regenerate the venv with the correct interpreter:

```
ubuntu@ip-10-0-0-190:~/learningsteps$ ls -la venv/bin/python*  
lrwxrwxrwx 1 root root 7 Sep 28 19:32 venv/bin/pytho -> python3  
lrwxrwxrwx 1 root root 16 Sep 28 19:32 venv/bin/python3 -> /usr/bin/python3  
lrwxrwxrwx 1 root root 7 Sep 28 19:32 venv/bin/python3.14 -> python3
```

venv troubleshooting

```
rm -rf venv  
/usr/bin/python3.14 -m venv venv  
source venv/bin/activate
```

8. SECURITY CONSIDERATIONS

Temporary Elastic IP on the database instance. Attaching an Elastic IP to `dbServer` provided outbound internet access for installing PostgreSQL, and was deleted immediately after installation to avoid both ongoing costs and an exposed attack surface. In production, the standard pattern is a **NAT Gateway** in the public subnet: the private instance gains outbound-only internet access while remaining unreachable from the internet at any point in time.

Layered network controls. This project relied solely on Security Groups (instance-level, stateful). AWS also offers **Network ACLs** (subnet-level, stateless), which act as an additional boundary layer. Defense in depth is achieved by combining both.

Secrets management. Database credentials were stored in a plain `.env` file on the instance. In production, AWS **Secrets Manager** or **Parameter Store** should be used to centralize and rotate credentials.

Restricted ingress. Port 8000 was opened to the internet (`0.0.0.0/0`) for testing. In a real deployment, this should be restricted to a load balancer, a known IP range, or placed behind proper HTTPS termination.

9. LESSONS LEARNED

Knowing how to work on one cloud provider makes it possible to work on others. It is mainly a matter of understanding each provider's **naming conventions**. More important still is understanding the **structure of a two- or three-tier architecture**: how networking works, subnets, isolation, and how to enable communication between machines, even when some are isolated from the internet, by accessing them through another machine in the architecture.